01 / PROBLEM AND CONTRIBUTION

APB interface for an FP8 accelerator

Advanced Digital Design & Verification · Lab 3 | Fall 2026 | Four-person team

RESULT AT A GLANCE

Implemented the accelerator's 16-bit APB interface and register map, with 320 output entries checked and 70 protocol checks in the reported Part 4 simulation.

The engineering question

How can a processor configure, launch, and monitor a matrix accelerator through a register interface?

Situation

The team's FP8 matrix engine needed a processor-facing interface for configuration, launch, completion, and exception reporting. Ruthvik's assigned contribution was the APB interface and register map.

Task

Wrap the existing engine with a 16-bit APB slave and verify register access, control sequencing, and result/status readback.

My contribution

Implemented Part 4: the APB interface and register map, configuration/status verification, integration documentation, and waveforms.

Tools and concepts

SystemVerilog / APB / Register-map design / VCS / Verdi

02 / DESIGN AND METHOD

From the task to implementation

01 / Implementation step

Added a 16-bit APB slave around the team's existing FP8 e3m4 matrix engine. The interface exposes matrix base addresses, strides, START, STATUS, and exception FLAGS.

02 / Implementation step

Used IDLE, PREPARE, and RUN control states to coordinate configuration and execution. DONE is asserted after output-memory writes complete. PREADY remains asserted in this no-wait-state interface.

03 / Implementation step

Verified normal and exception cases, non-zero base addresses, and non-unit strides with APB read/write tasks.

Software-visible control

The register map provides a boundary between configuration/status access and the existing computation engine. Base addresses and strides let the testbench exercise different memory locations without claiming a new arithmetic datapath contribution.



APB register-write sequence

Annotated protocol waveform from Part 4, the APB interface work.

03 / RESULTS AND ENGINEERING PERSPECTIVE

Evidence, interpretation, and next steps

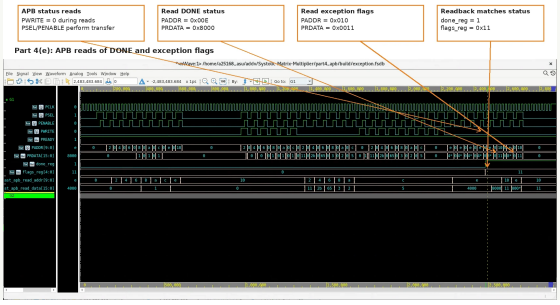
APB data width	16 bits
Outputs checked	320
Protocol checks	70

Reported result

The Part 4 simulation reported 320 output entries checked and 70 APB protocol checks. The overflow example returned STATUS = 0x8000 and FLAGS = 0x0011 after completion.

Completion and exceptions

The no-wait-state interface keeps PREADY asserted. Completion follows the output writes; status and exception flags are then read through APB. Checking transaction sequencing complements numerical output comparison.



Result and status readback

Annotated APB readback waveform from the report.

Scope and limitations

The arithmetic units and array tiles were developed by teammates. Array synthesis estimates in the report are pre-layout and still contain hold/capacitance violations; this case study does not claim physical timing closure.

Possible next steps - proposed, not completed results

- Publish the register map and a concise APB transaction example alongside shareable RTL.
- Address reported implementation violations before making physical timing-closure claims.

Source material: Team Lab 3 report, pp. 15-20; contribution allocation on p. 20.

Prepared from the supplied reports, project files, and verified portfolio copy. Figures retain their original content; explanatory diagrams are labelled.